

Chapter 4

The life of a scene

What Phaser calls, and when - and the scene manager's other powers

DemoScene

update() calls: 106
game steps seen: 106
time: 2593 ms
this.time.now: 2593 ms
delta: 16.6 ms



```
- constructor()
0 init()
0 preload()
2 loader: complete
2 create()
2 CREATE event
2 PRE UPDATE event
2 Spinner.preUpdate()
2 UPDATE event
2 update()
2 POST UPDATE event
2 RENDER event
```

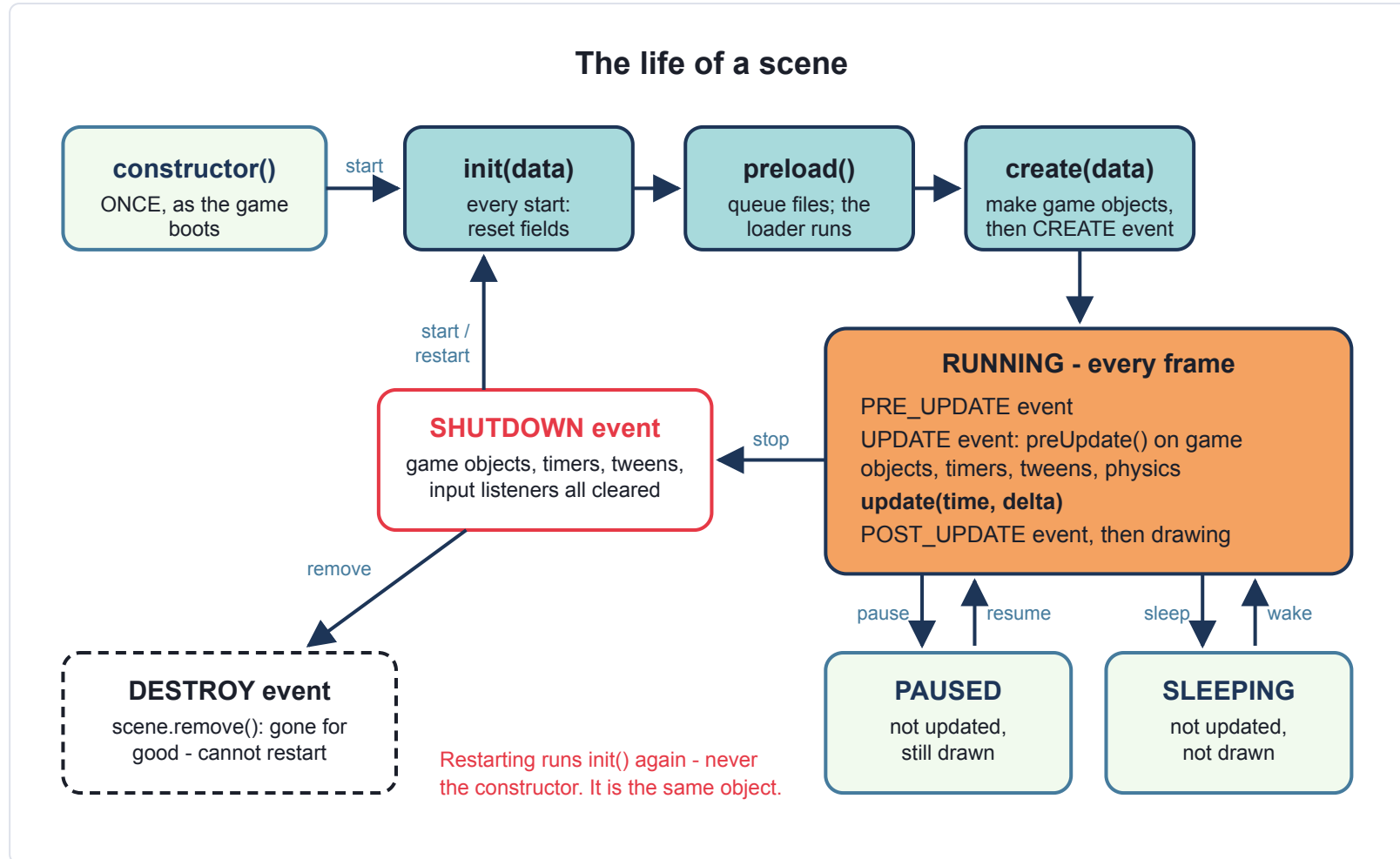
DemoScene: RUNNING isActive: yes isPaused: no isSleeping: no isVisible: yes
fps: 56.1 listeners for CREATE: 1 for the game's STEP: 1 leak mode: off super.preUpdate(): called

1 pause 2 resume 3 sleep 4 wake 5 restart 6 stop 7 launch 8 remove
L leak mode on/off S skip super.preUpdate() on/off C clear the log

Today

- the order: `constructor`, `init`, `preload`, `create`, `update`
- one frame, step by step; scene **events**
- a `Sprite` 's own `preUpdate` - and `super`
- `launch`, `pause / resume`, `sleep / wake`, `stop`, `remove`
- cleaning up: the listeners Phaser does **not** remove
- `time` vs `delta`; the frame rate
- a HUD and a pause menu, as scenes

The life of a scene



Once, or every start?

```
constructor() {  
    super(DEMO_SCENE);  
    eventLog.add("constructor()");  
}  
  
// Runs every time the scene starts: the place to reset fields  
init(): void {  
    this.log("init()");  
    this.updates = 0;  
    ...  
}
```

- constructor: **once**, when the game boots
- `init()` , `preload()` , `create()` : **every** start - then the `CREATE` event
- restart: `init()` again, never the constructor. Same object

Try it: Lifecycle Logger

- build, serve, open `ch04_lifecycle_logger`
- read the log from the top: what happens in frame 0? frame 2?
- press **5** (restart): what is missing from the log this time?
- press **S**: what does `super.preUpdate()` do?

DemoScene

update() calls: 106
game steps seen: 106
time: 2593 ms
this.time.now: 2593 ms
delta: 16.6 ms

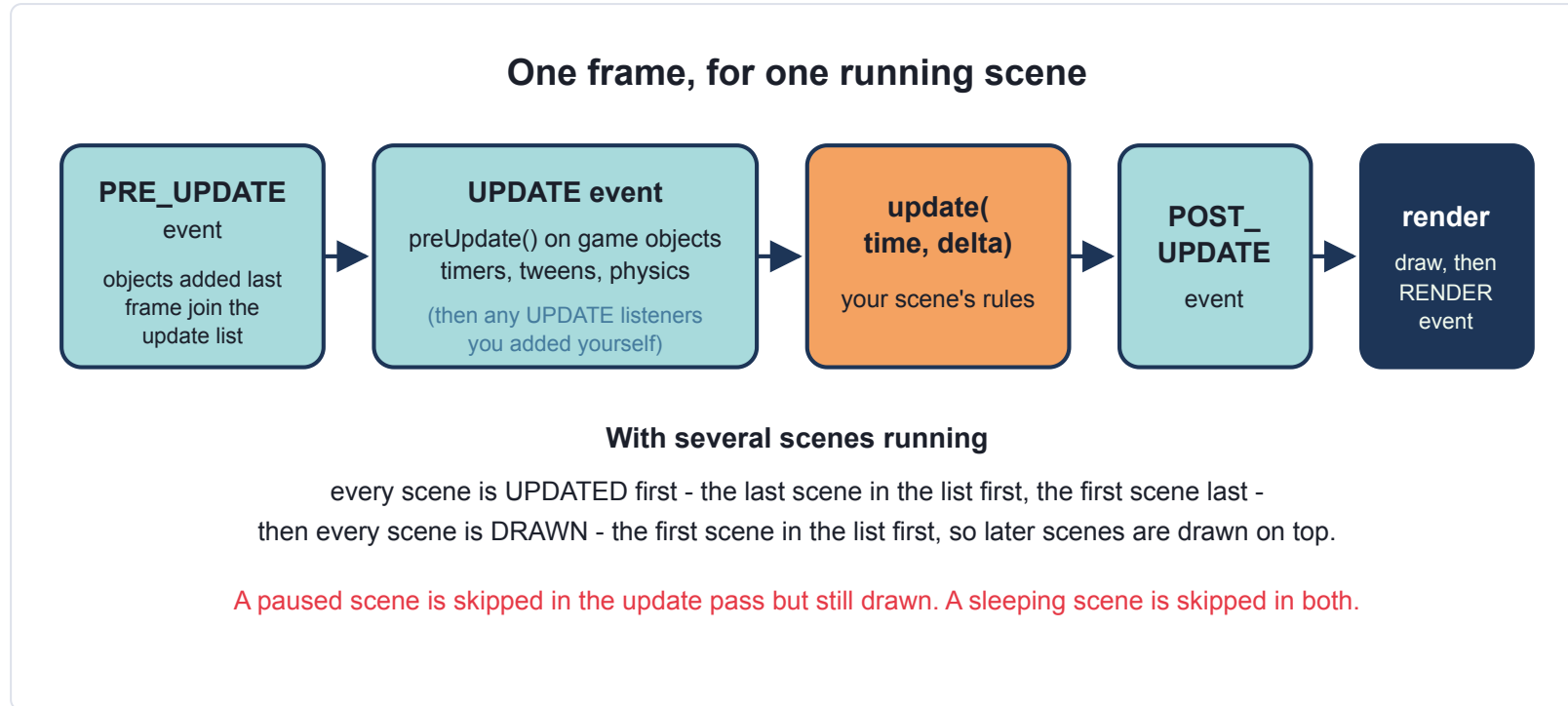


```
- constructor()
0 init()
0 preload()
2 loader: complete
2 create()
2 CREATE event
2 PRE UPDATE event
2 Spinner.preUpdate()
2 UPDATE event
2 update()
2 POST UPDATE event
2 RENDER event
```

DemoScene: RUNNING isActive: yes isPaused: no isSleeping: no isVisible: yes
fps: 56.1 listeners for CREATE: 1 for the game's STEP: 1 leak mode: off super.preUpdate(): called

1 pause 2 resume 3 sleep 4 wake 5 restart 6 stop 7 launch 8 remove
L leak mode on/off S skip super.preUpdate() on/off C clear the log

One frame, in order



`update()` comes after every game object has moved itself.

Listening for scene events

```
this.events.on(Phaser.Scenes.Events.PAUSE, this.onPause, this);
this.events.on(Phaser.Scenes.Events.RESUME, this.onResume, this);
...
this.game.events.on(Phaser.Core.Events.STEP, this.onGameStep, this);
...
this.events.once(Phaser.Scenes.Events.SHUTDOWN, this.onShutdown, this);
```

- `on(event, method, context)` - the **context** is the object to call it on
- Java: `this::onPause` remembers its object. JavaScript's `this.onPause` does not
- the same three arguments take it off again: `off(event, method, context)`

A Sprite's own preUpdate

Coin.ts

```
protected override preUpdate(time: number, delta: number): void {  
    super.preUpdate(time, delta);  
    ...  
    this.age = this.age + delta;  
    ...  
}
```

- Image has no preUpdate - Sprite does: it moves the animation on
- override and protected - the compiler checks
- super.preUpdate(...) - **nobody** checks. Leave it out: moves, never animates

The scene manager

	Updated?	Drawn?	Game objects
<code>launch(key)</code> - alongside this one	yes	yes	new
<code>pause(key)</code> / <code>resume(key)</code>	no	yes	kept
<code>sleep(key)</code> / <code>wake(key)</code>	no	no	kept
<code>stop(key)</code>	no	no	destroyed
<code>remove(key)</code>	-	-	gone for good

`start` stops **the caller**. `restart()` has no key. Requests are **queued**: they happen as the scene manager's next update begins.

Paused or asleep?

- **paused**: frozen, still on screen
- **asleep**: not drawn either
- both keep everything
- neither hears the keyboard - so the keys live in **another** scene

DemoScene is
not being drawn

```
- constructor()
0 init()
0 preload()
2 loader: complete
2 create()
2 CREATE event
2 PRE UPDATE event
2   Spinner.preUpdate()
2   UPDATE event
2   update()
2   POST UPDATE event
2   RENDER event
112 >> scene.pause(DEMO_SCENE)
112 PAUSE event
148 >> scene.resume(DEMO_SCENE)
148 RESUME event
148 PRE UPDATE event
148   Spinner.preUpdate()
148   UPDATE event
148   update()
148   POST UPDATE event
148   RENDER event
174 >> scene.sleep(DEMO_SCENE)
174 SLEEP event
```

DemoScene: SLEEPING isActive: no isPaused: no isSleeping: yes isVisible: no
fps: 53.1 listeners for CREATE: 1 for the game's STEP: 1 leak mode: off super.preUpdate(): called

1 pause 2 resume 3 sleep 4 wake 5 restart 6 stop 7 launch 8 remove
L leak mode on/off S skip super.preUpdate() on/off C clear the log

Cleaning up

When a scene shuts down, Phaser clears:

- its game objects, timers, tweens, input listeners

It does **not** clear listeners on:

- the scene's own `this.events` (the object - and emitter - is reused)
- `this.game.events`, `this.registry.events`, another scene's `events`

Whatever you add to an emitter that outlives your scene, take off in `SHUTDOWN` .

A leak you can see

Leak mode (L), then restart twice (5, 5):

- every event logged **three** times
- "game steps seen" climbs 3x too fast
- 3 listeners for `CREATE`, 3 for `STEP`

In a game: doubled scores, tripled sounds... then a crash.

DemoScene

update() calls: 68
game steps seen: 201
time: 7776 ms
this.time.now: 7776 ms
delta: 20.0 ms



```
244 POST_UPDATE event
244 POST_UPDATE event
244 RENDER event
263 >> demo.scene.restart()
263 SHUTDOWN event
263 leak mode: listeners left behind!
263 init()
263 preload()
263 loader: complete
263 create()
263 CREATE event
263 CREATE event
263 CREATE event
263 PRE_UPDATE event
263 PRE_UPDATE event
263 PRE_UPDATE event
263 UPDATE event
263 UPDATE event
263 Spinner.preUpdate()
263 UPDATE event
263 update()
263 POST_UPDATE event
263 POST_UPDATE event
263 POST_UPDATE event
263 RENDER event
```

DemoScene: RUNNING isActive: yes isPaused: no isSleeping: no isVisible: yes
fps: 45.8 listeners for CREATE: 3 for the game's STEP: 3 leak mode: ON super.preUpdate(): called

1 pause 2 resume 3 sleep 4 wake 5 restart 6 stop 7 launch 8 remove
L leak mode on/off S skip super.preUpdate() on/off C clear the log

Time and delta

- `time` / `this.time.now` - the **game's** clock. Keeps going while a scene is paused
- `delta` - ms since the last frame. Only arrives while the scene **runs**
- timers (`this.time.delayedCall`) belong to the scene: they wait during a pause

```
fps: {  
  target: 60,  
  limit: 0,  
},
```

`limit: 10` - the coin moves just as fast, in bigger steps

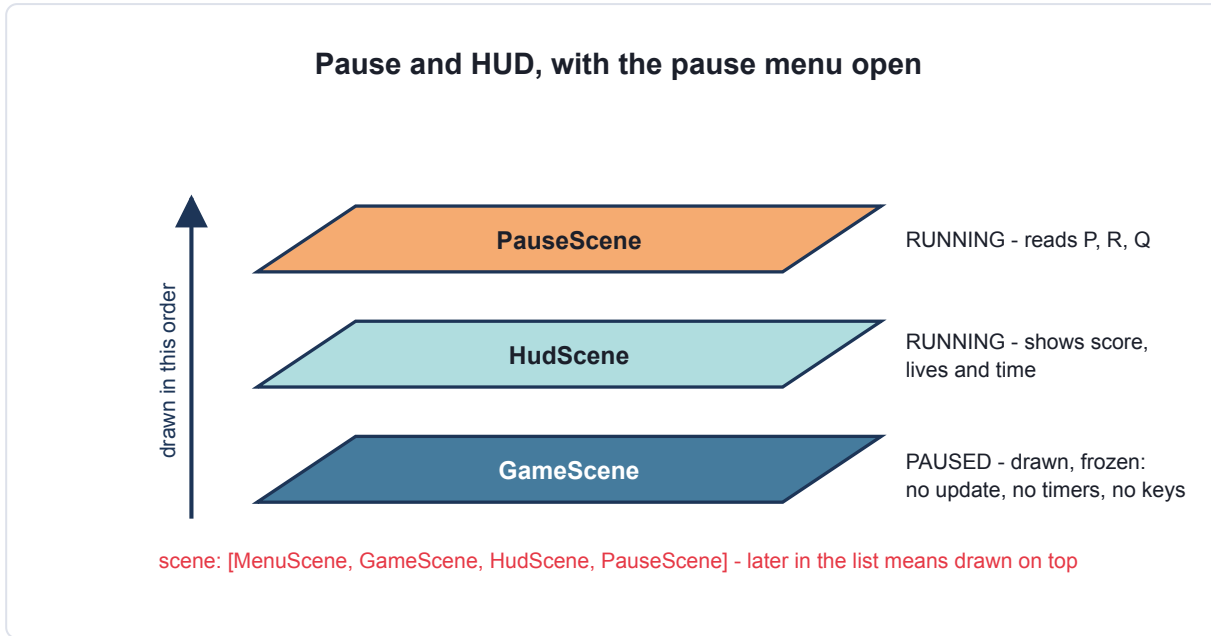
Try it: Pause and HUD

```
scene: [MenuScene, GameScene, HudScene, PauseScene],
```

- arrows, collect coins, avoid triangles
- P pauses; R, Q from the menu
- which scenes are running when paused?



Scenes side by side



```
const hudData: HudData = { score: this.score, lives: this.lives, seconds: this.secondsShown };  
this.scene.launch(HUD_SCENE, hudData);
```

The HUD listens

```
this.events.emit(SCORE_CHANGED, this.score);
```

```
const gameScene = this.scene.get(GAME_SCENE);
gameScene.events.on(SCORE_CHANGED, this.showScore, this);
...
this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
  gameScene.events.off(SCORE_CHANGED, this.showScore, this);
  ...
});
```

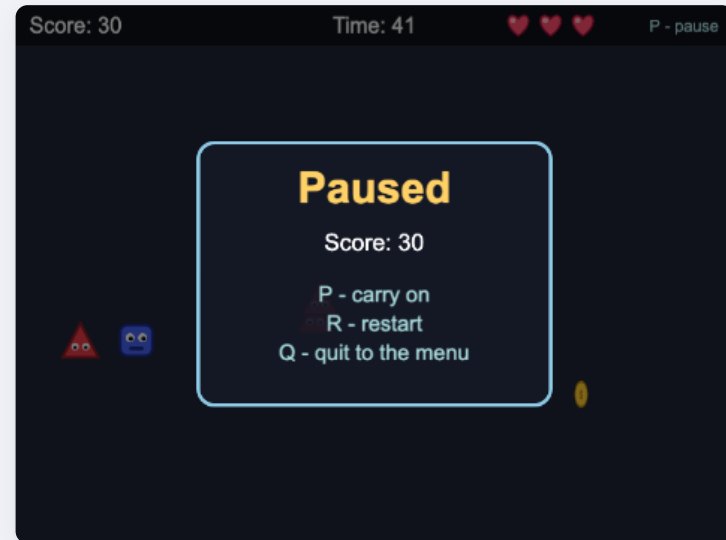
GameScene emits; HudScene listens. GameScene's emitter outlives the HUD - so the HUD cleans up.

Pausing

```
private pauseGame(): void {  
    const data: PauseData = { gameOver: false, title: "Paused", score: this.  
    this.scene.launch(PAUSE_SCENE, data);  
    this.scene.pause();  
}
```

```
private carryOn(): void {  
    this.scene.resume(GAME_SCENE);  
    this.scene.stop();  
}
```

GameScene pauses itself; PauseScene (running) resumes it



Time that survives a pause

```
this.player.setAlpha(0.4);  
this.time.delayedCall(SAFE_TIME, () => {  
    this.safe = false;  
    this.player.setAlpha(1);  
});
```

Not `this.safeUntil = this.time.now + SAFE_TIME - this.time.now` keeps going during a pause. Measure time with **timers** and `delta` (as `Coin` ages itself).

Summary

- constructor **once**; `init` , `preload` , `create` every start; `update` every frame
- frame: `PRE_UPDATE` , `UPDATE` (objects' `preUpdate`) , `update()` , `POST_UPDATE` , `draw`
- `Sprite` subclass: `protected override preUpdate + super.preUpdate(time, delta)`
- `launch` alongside; `pause` = frozen but drawn; `sleep` = hidden too; `stop` ; `remove`
- scenes are drawn in list order; `bringToTop`
- clean up listeners on long-lived emitters in `SHUTDOWN`
- pausable time: timers and `delta`

Challenges

1. **Count the frames drawn** - `RENDER` events vs `update()` calls
2. **Thirty frames a second** - `fps.limit`, and a real frame counter on the HUD
3. **How long was I away?** - time paused, counted once
4. **Settings** - a settings scene; the pause menu sleeps; H hides the HUD
5. **Fade in, fade out** - camera fades between scenes
6. **Auto-pause, without a leak** - `BLUR` / `HIDDEN`, proved leak-free

Next: **Chapter 5 - Audio**